Dynamic Graph Algorithms

CSC-531

Incremental DFS in DAGs

Dr. Shahbaz Khan
Department of Computer Science and Engineering,
Indian Institute of Technology Roorkee
shahbaz.khan@cs.iitr.ac.in

Questions

Think about **incremental maintenance of DFS** trees.

Directed acyclic graph

(Think of using changes in Topological order)

Det A_m

Det $O(m)$ ✗

Rand. $O(m)$

Undirected

(Think of ensuring increase of distance from root)

Directed graph

(Think of possible difficulty)

No non-trivial algorithm exists for $O(m)$ WC or A_m

Partial cases

Even Randomized

Depth First Search Trees

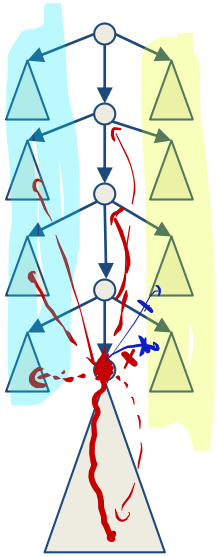
Tree edges and Non-Tree edges

1. For a DFS tree of a general directed graph, the non-tree edges can be (0 point)

   53% of respondents (17 of 32) answered this question correctly.

More Details

- From a vertex to all its ancestors 31 ✓
- From a vertex to all its descenda... 25 ✓
- From a vertex to any vertex on i... 27 ✓
- From a vertex to any vertex on i... 10

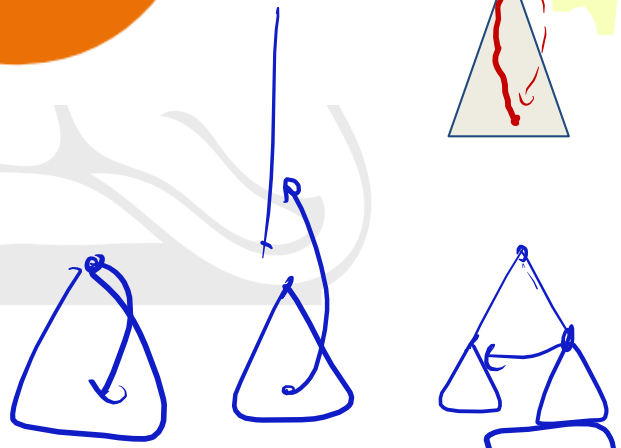


From a vertex to all its ancestors ✓

From a vertex to all its descendants ✓

From a vertex to any vertex on its left in the tree ✓

From a vertex to any vertex on its right in the tree



Non-Tree edges?

General Graph: Forward Edges, Back Edges and Cross Edges [R->L]

Depth First Search Trees

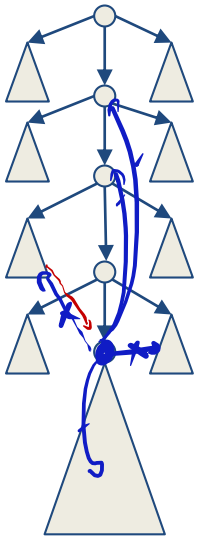
Tree edges and Non-Tree edges

2. For a DFS tree of an undirected graph, the non-tree edges can be (0 point)

31% of respondents (10 of 32) answered this question correctly.

More Details

- From a vertex to all its ancestors 25 ✓
- From a vertex to all its descenda... 11 ✓
- From a vertex to any vertex on i... 10
- From a vertex to any vertex on i... 7



From a vertex to all its ancestors ✓

From a vertex to all its descendants ✓

From a vertex to any vertex on its left in the tree

From a vertex to any vertex on its right in the tree

Non-Tree edges?

Undirected Graph: Forward Edges, Back Edges

Depth First Search Trees

Tree edges and Non-Tree edges

3. For a DFS tree of a directed acyclic graph, the non-tree edges can be (0 point)
41% of respondents (13 of 32) answered this question correctly.

More Details

- From a vertex to all its ancestors 8
- From a vertex to all its descenda... 21 ✓
- From a vertex to any vertex on i... 25 ✓
- From a vertex to any vertex on i... 10

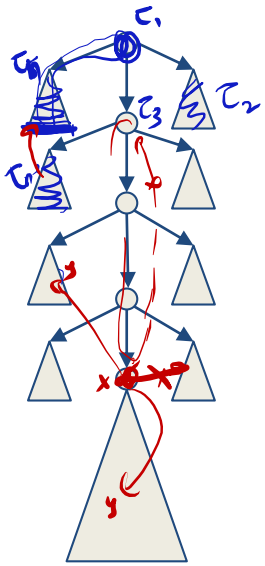
From a vertex to all its ancestors

From a vertex to all its descendants ✓

From a vertex to any vertex on its left in the tree ✓

From a vertex to any vertex on its right in the tree

Reverse finish time
Post order traversal



for directed acyclic graph the valid edges was a bit unclear and it only mentions cross edges a explanation on that will be helpful.

Non-Tree edges?

DAGs: Forward Edges, Cross Edges [R-> L]

Depth First Search Trees

4. Briefly describe why topological ordering is reverse of DFS Finish Time (Postorder) (0 point)

5. Describe a simple algorithm for maintaining Topological ordering in decremental setting

Trivial

Removing an edge would only reduce dependency of some node on another node. The original Reverse DFS ordering will still be valid as the connections are only removed meaning some nodes may be executed before they were earlier but doing them at the same time as previously would still be valid.

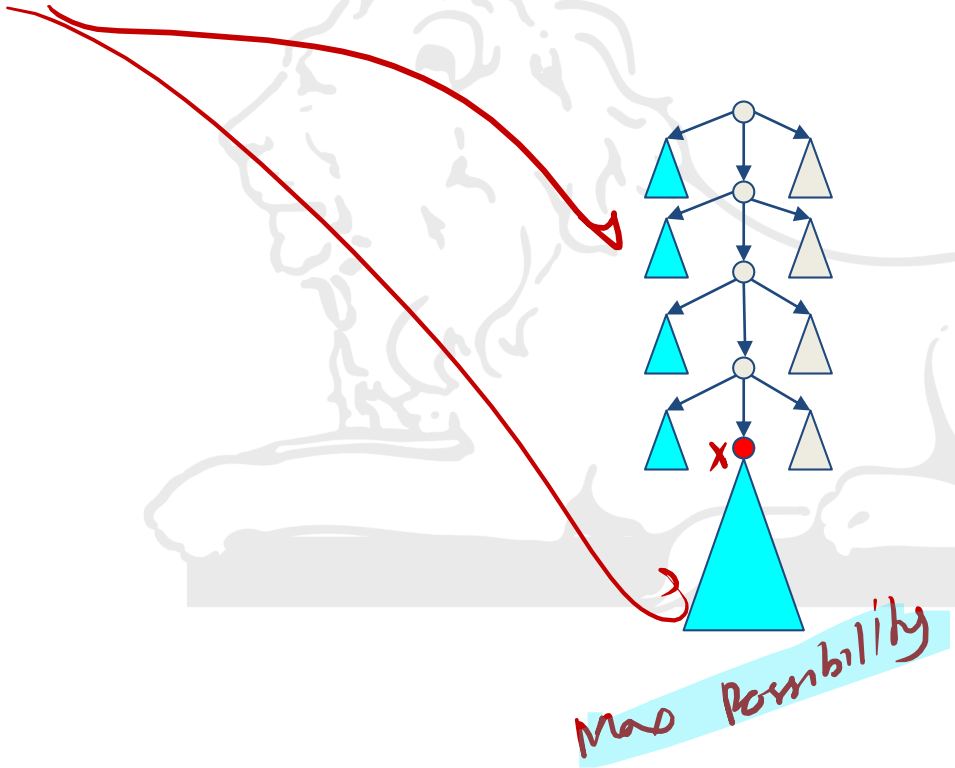
I don't think the original topological order would in any case violating any new graph obtained by removing edges. This is because there would never be any nodes u and v in the graph such that u appears before v in the topological order and there is a path from v to u in the graph. This is because we never add edges but only remove edges. So the original topological order would always be valid in the decremental setting.

Incremental Depth First Search Trees (DAG)

Given a DAG G, describe an algorithm to maintain its DFS under insertion of edges

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L



Incremental Depth First Search Trees (DAG)

Given a DAG G, describe an algorithm to maintain its DFS under insertion of edges

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

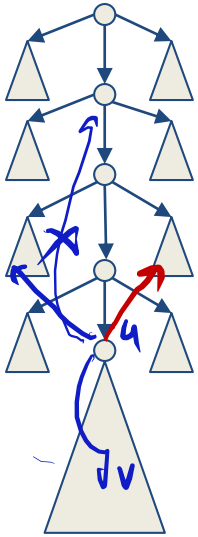
Insertion of Edge (u,v)?

Ancestor to Descendant? X

Descendant to Ancestor? NA

Cross R->L? X

Cross L->R? PROCESS



Incremental Depth First Search Trees (DAG)

Given a DAG G, describe an algorithm to maintain its DFS under insertion of edges

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

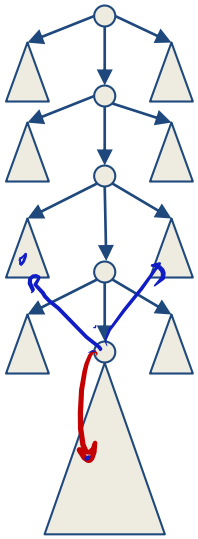
Insertion of Edge (u,v)?

1 Ancestor to Descendant? → DFSF[u] < DFSF[v] X

~~Descendant to Ancestor?~~

2 Cross R->L?

3 Cross L->R?



Detection?

Using DFS Visit Time

1 DFSV[u] < DFSV[v]

2 DFSV[u] > DFSV[v]

3 DFSV[u] < DFSV[v]

Using DFS Finish Time

1 DFSF[u] > DFSF[v]

2 DFSF[u] > DFSF[v]

3 DFSF[u] < DFSF[v] ⇒ τ(u) > τ(v)

Incremental Depth First Search Trees (DAG)

Given a DAG G, describe an algorithm to maintain its DFS under insertion of edges

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

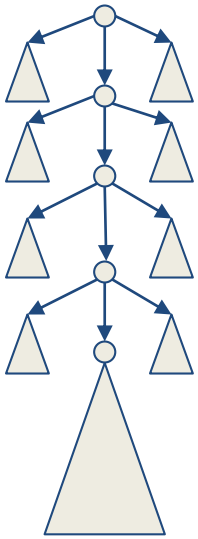
Insertion of Edge (u,v)?

Ancestor to Descendant?

~~Descendant to Ancestor?~~

Cross R->L?

Cross L->R?



Detection?

Using DFS Visit Time

Using DFS Finish Time

DFS Finish[u] < DFS Finish[v]

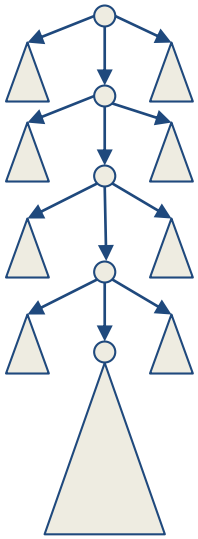
ALTERNATE VIEW?

Topological Ordering

Topological Order:

An ordering top[-] over vertices in G such that

If **edge (u,v) in E**, then **top[u] < top[v]**



Topological Ordering

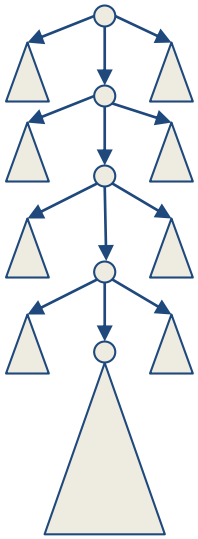
Topological Order:

An ordering top[-] over vertices in G such that

If **edge (u,v) in E**, then **top[u] < top[v]**

Possible Algorithm:

Reverse DFS Finish Time

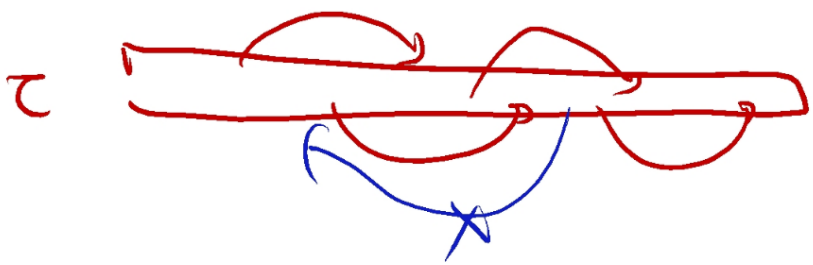


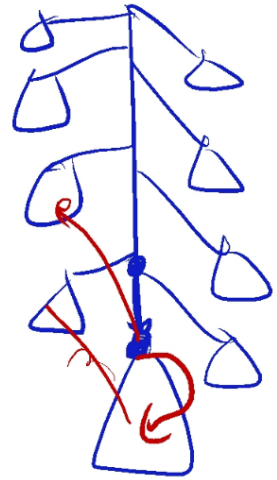
Topological Order of $G(V,E)$

$\tau : V \to [n]$

[such that $(u,v) \in E$
$\tau(u) < \tau(v)$]

only for DAG



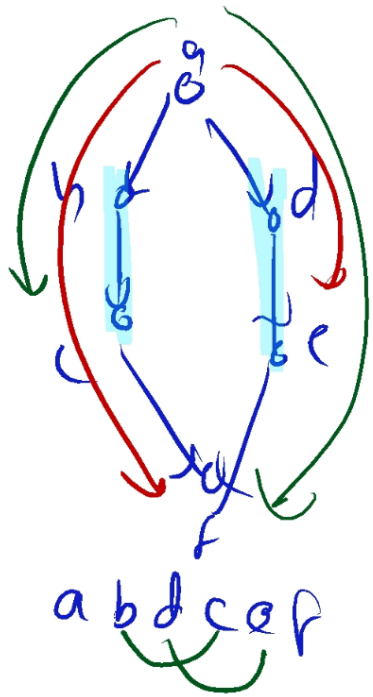


(u,v)

Tree edge ✓

Forward " ✓

Cross edge ✓



Reverse finish time of DFS tree is valid τ

post order traversal

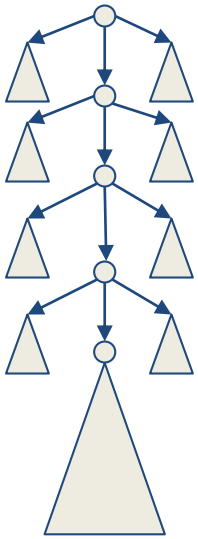
RECURSIVE TOP ORDER → DFS tree computes

→ Dec. X

Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L



Insertion of Edge (u,v)?

Ancestor to Descendant?

~~Descendant to Ancestor?~~

Cross R->L?

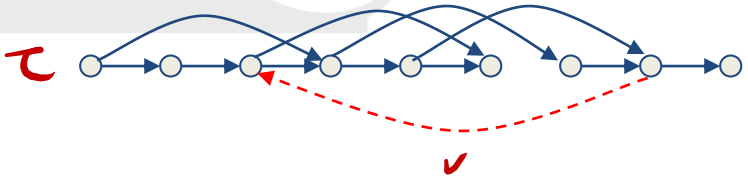
Cross L->R?

Detection?

Using DFS Visit Time

Using DFS Finish Time

DFS Finish[u] < DFS Finish[v]



Topological Order (Reverse DFS Finish Time)

Top[u] > Top[v]

Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge (u,v)?

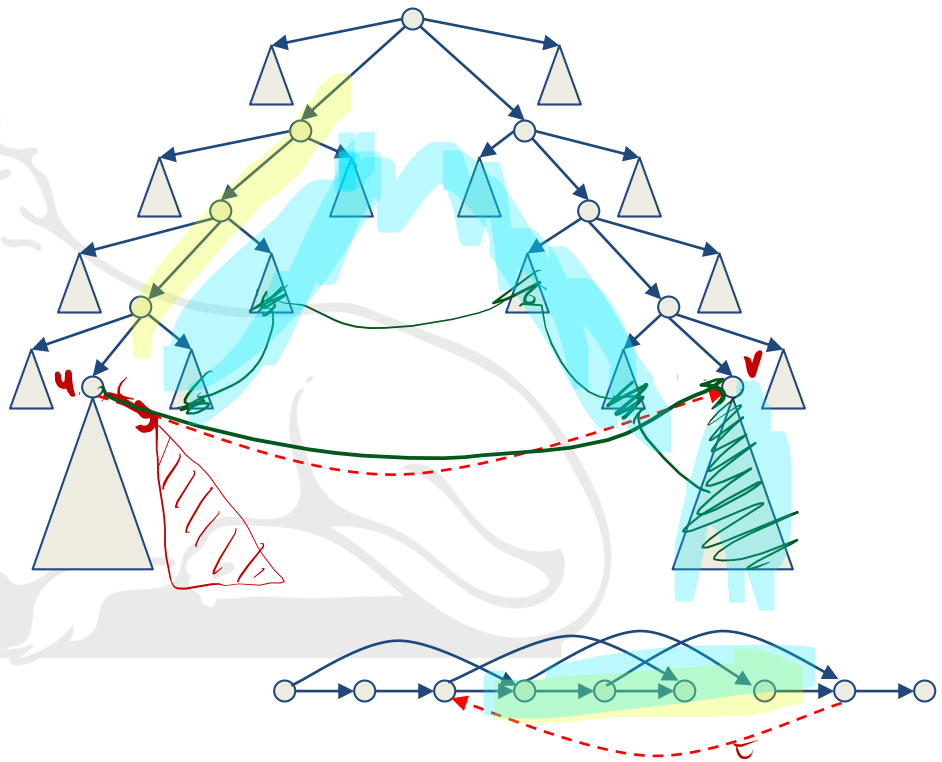
Cross L->R?

Detection?

DFS Finish[u] < DFS Finish[v]

Top[u] > Top[v]

Affected Vertices?



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge (u,v)?

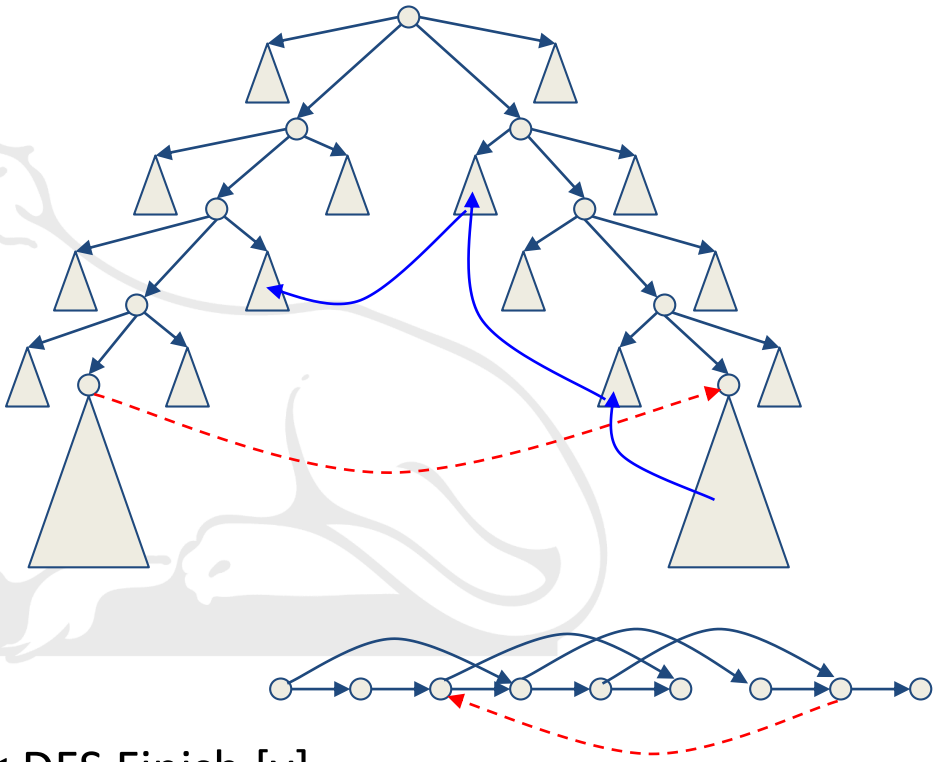
Cross L->R?

Detection?

DFS Finish[u] < DFS Finish[v]

Top[u] > Top[v]

Affected Vertices?



DFS Finish[u] < DFS Finish[x] < DFS Finish [v]

Top[u] > Top[x] > Top[v]

Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

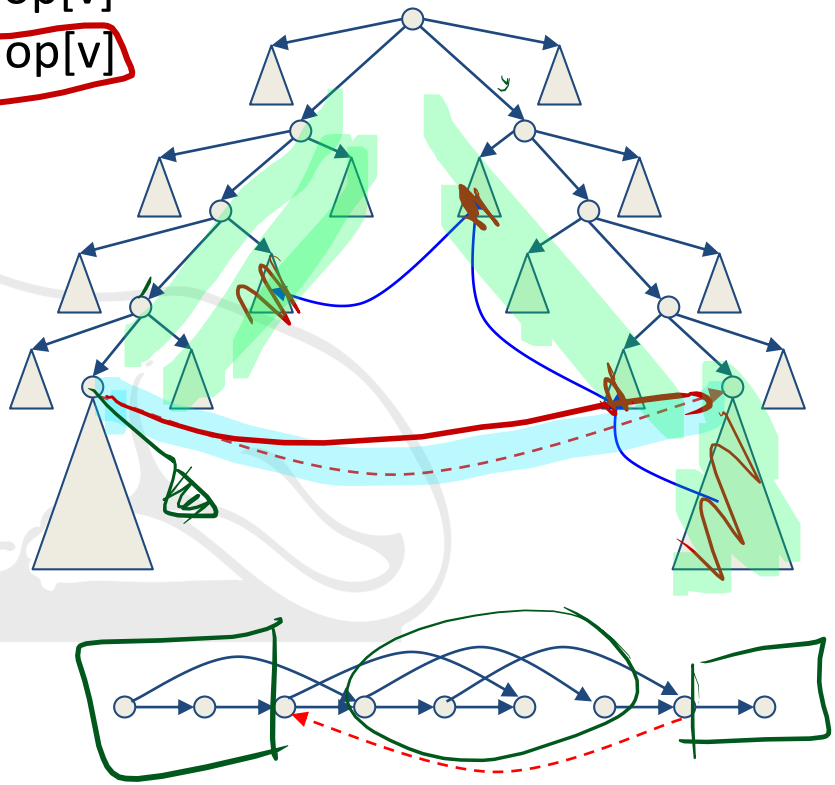
Insertion of Edge(u,v)? Cross L->R

Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Approach:

```
Insert(u,v){
    If(Top[u]> Top[v]){
            Add (u,v) in T,
        Partial DFS from v in Affected Vertices;
        Update Topological Order;
    }
}
```



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

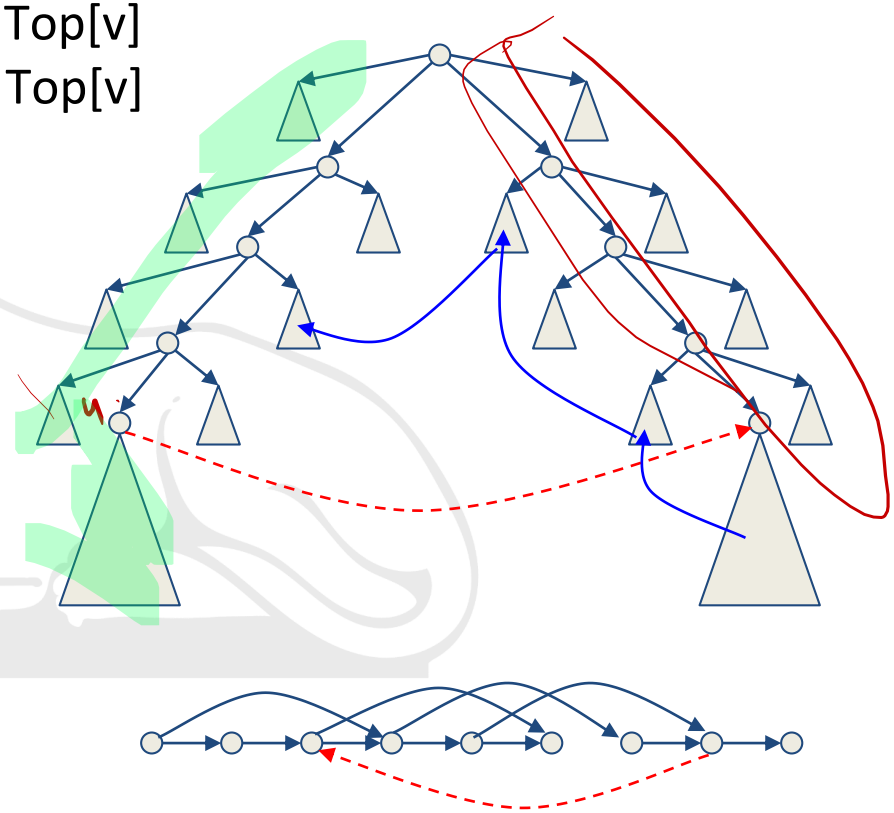
Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Algorithm:

```
Insert(u,v){
    If(Top[u]> Top[v]){
        Add (u,v) in T,
        Set visit[-]=0 for all vertices
        Partial DFS(v,u);
        Update Topological Order;
}}

PartialDFS(v,u){
        visit[v]=1;
        ForAll( (v,w) in E){
            If(Top[w] < Top[u] and visit[w]=0)
                Add (v,w) in T;
                Partial DFS(w,u);
}
```



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

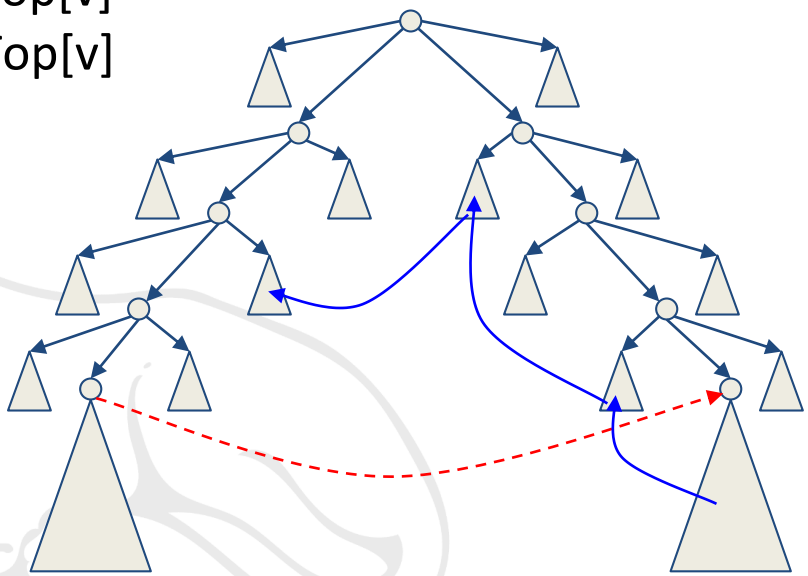
Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Algorithm:

```
Insert(u,v){
      If(Top[u]> Top[v]){
                  Add (u,v) in T,
            Set visit[-]=0 for all vertices
            Partial DFS(v,u);
            Update Topological Order;
}}

PartialDFS(v,u){
      visit[v]=1;
      ForAll( (v,w) in E){
                  If(Top[w] < Top[u] and visit[w]=0)
                        Add (v,w) in T;
                        Partial DFS(w,u);
}
```



EXAMPLE?

Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

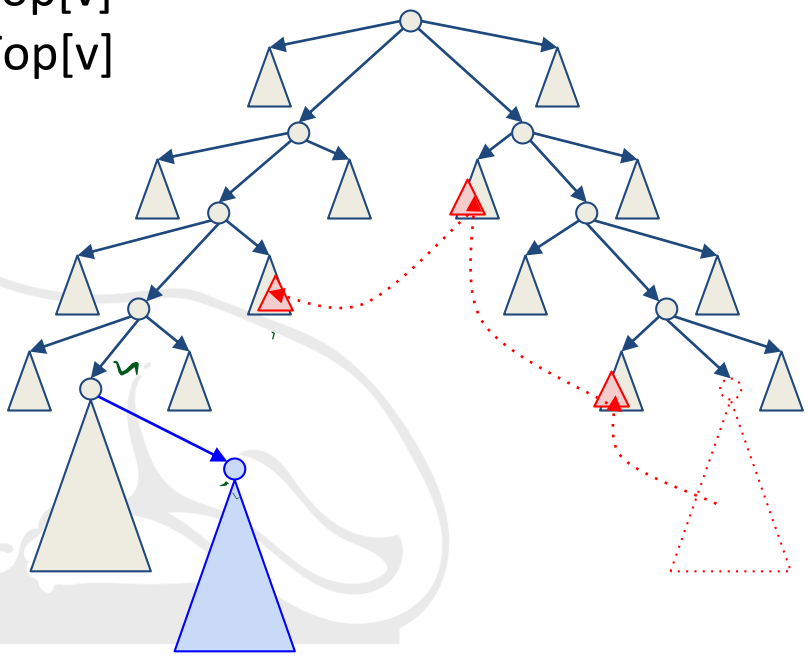
Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Algorithm:

```
Insert(u,v){
      If(Top[u]> Top[v]){
                  Add (u,v) in T,
            Set visit[-]=0 for all vertices
            Partial DFS(v,u);
            Update Topological Order;
}}

PartialDFS(v,u){
      visit[v]=1;
      ForAll( (v,w) in E){
                  If(Top[w] < Top[u] and visit[w]=0)
                        Add (v,w) in T;
                        Partial DFS(w,u);
}
```



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

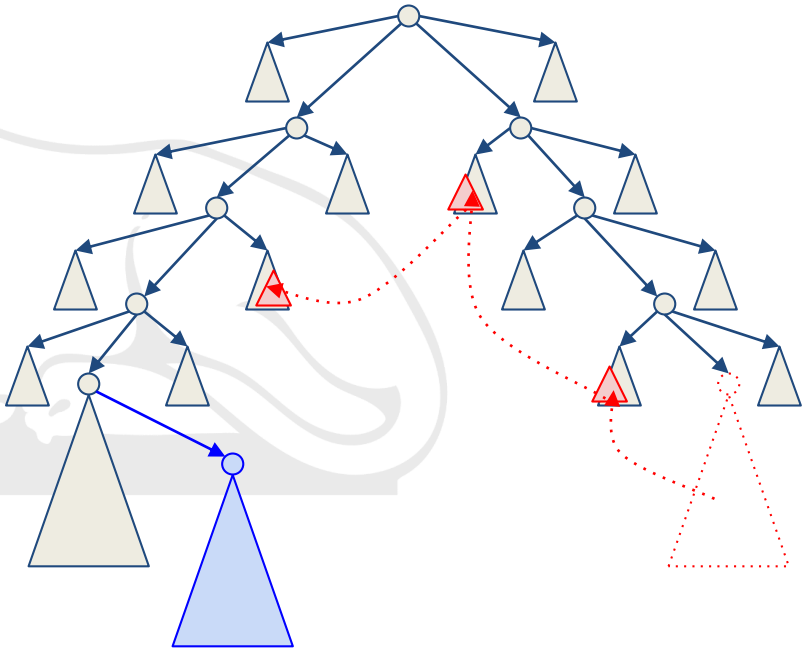
Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Correctness?



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

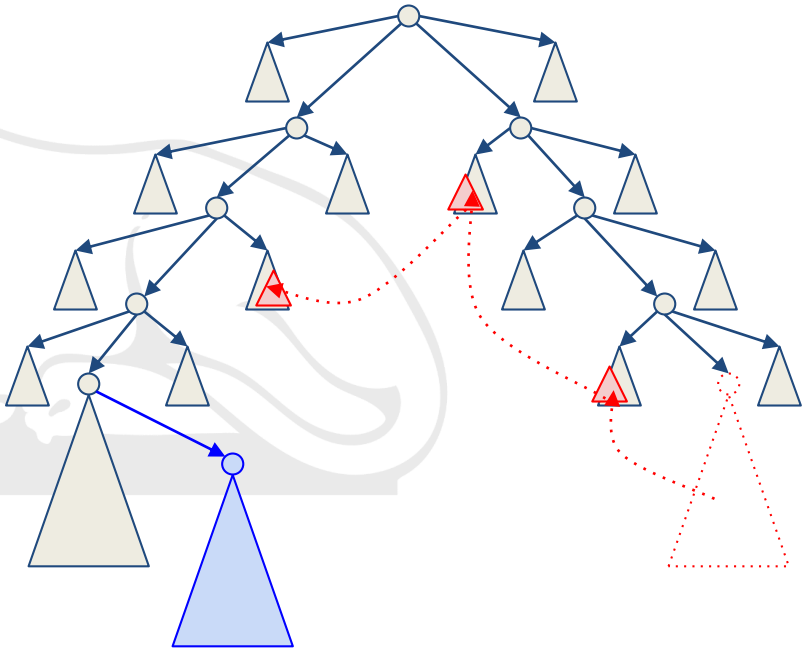
Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Correctness:

No Back edge

No Cross edge [L->R]?

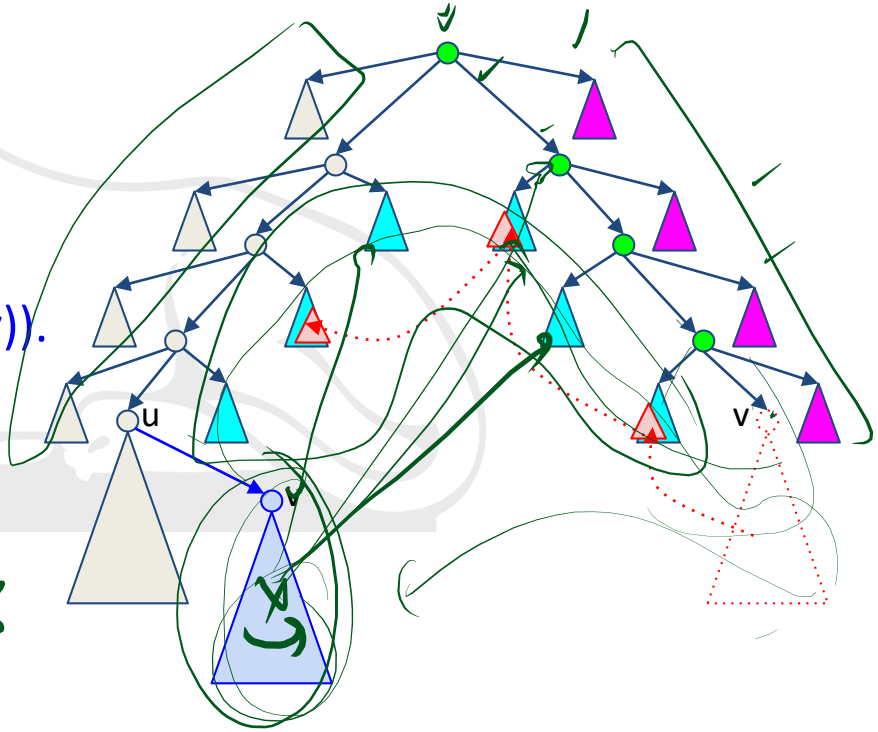


Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

Correctness:
No Back edge (DAG)
No Cross edge [L->R]?
Assume a cross edge (x,y) exists [L->R]
Either x, y or both were affected (in new T(v)).
Vertices moved left, so x surely in new T(v).

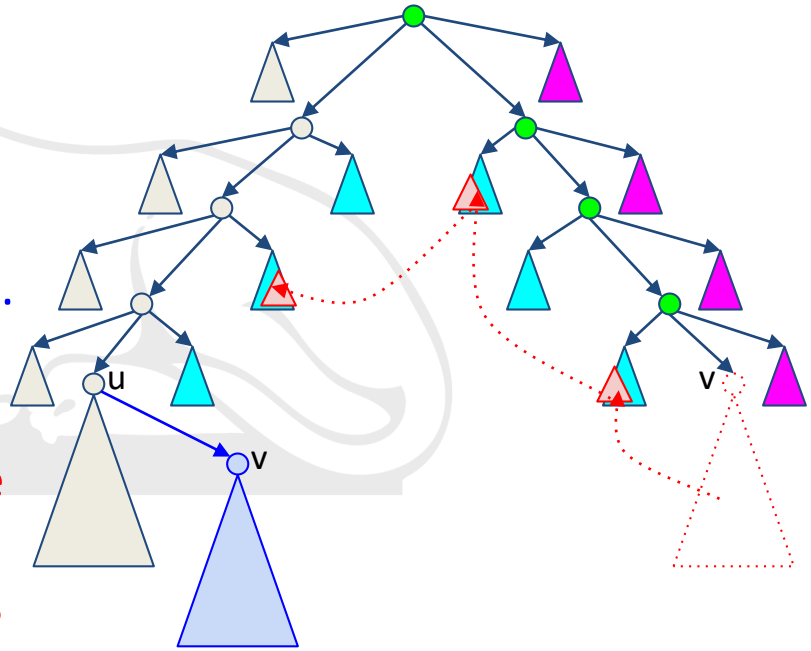
y can right of old ancestors of v?
y can be old ancestor of v?
y can be in Affected region?
y can be in T(v)?



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

Correctness:
No Back edge (DAG)
No Cross edge [L->R]?
Assume a cross edge (x,y) exists [L->R]
Either x, y or both were affected (in new T(v)).
Vertices moved left, so x surely in new T(v).



y can right of old ancestors of v? X Cross Edge
y can be old ancestor of v? X DAG
y can be in Affected region? X Partial DFS
y can be in T(v)? X Partial DFS

Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]

Reachability? Ancestor to Descendant + R-> L

Insertion of Edge(u,v)? Cross L->R

Detection Top[u] > Top[v]

Affected Vertices? Top[u] > **Top[x]** > Top[v]

Analysis:

Set visit[-]?

Update Topological Order?

Partial DFS?

O(n)

```
Insert(u,v){
    If(Top[u]> Top[v]){
            Add (u,v) in T;
        Set visit[-]=0 for all vertices
        Partial DFS(v,u);
        Update Topological Order;
}}

PartialDFS(v,u){
    visit[v]=1;
    ForAll( (v,w) in E){
            If(Top[w] < Top[u] and visit[w]=0)
                Add (v,w) in T;
                Partial DFS(w,u);
}
```

O(1)

O(n)

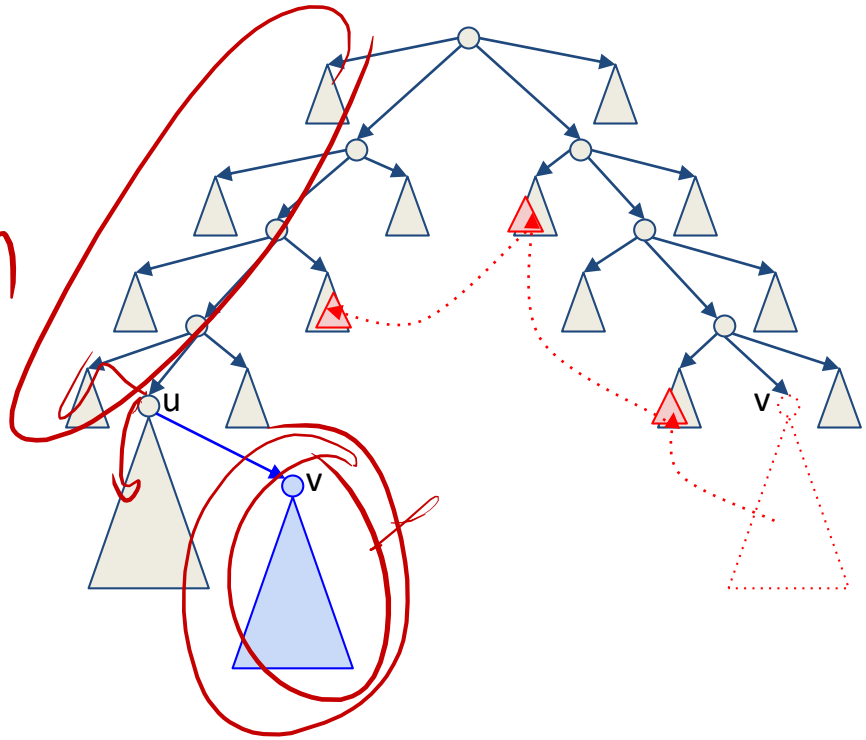
Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

Analysis:
Set visit[-] O(n)
Update Topological Order? O(n)
Partial DFS $k \sum deg(v)$ → wc O(m)
=> Sum of Degree of new T(v)

What changed for T(v)?

Think?
All pairs reachability
O(mn)?



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

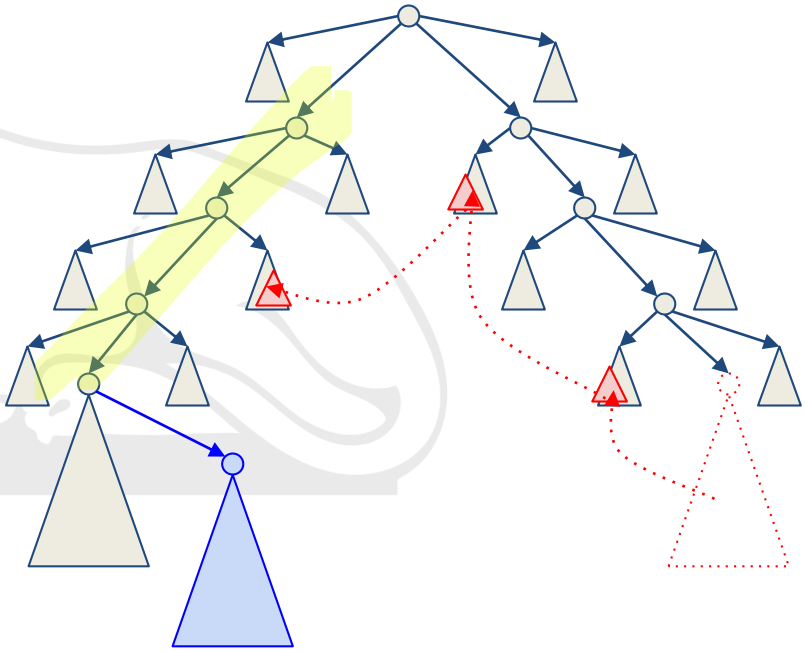
Analysis:
Set visit[-] O(n)
Update Topological Order? O(n)
Partial DFS
=> Sum of Degree of new T(v)

Reachability of T(v) from u?

freshly reachable from u

$\sum \deg(v)$

$v \in T(v)$



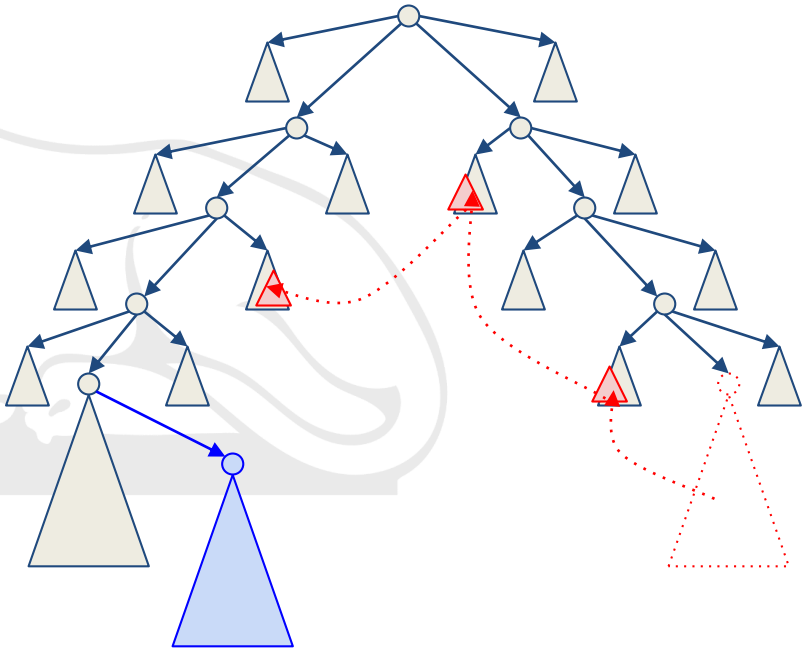
Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

Analysis:
Set visit[-] O(n)
Update Topological Order? O(n)
Partial DFS
=> Sum of Degree of new T(v)

Reachability of T(v) from u?
T(v) newly reachable from at least u

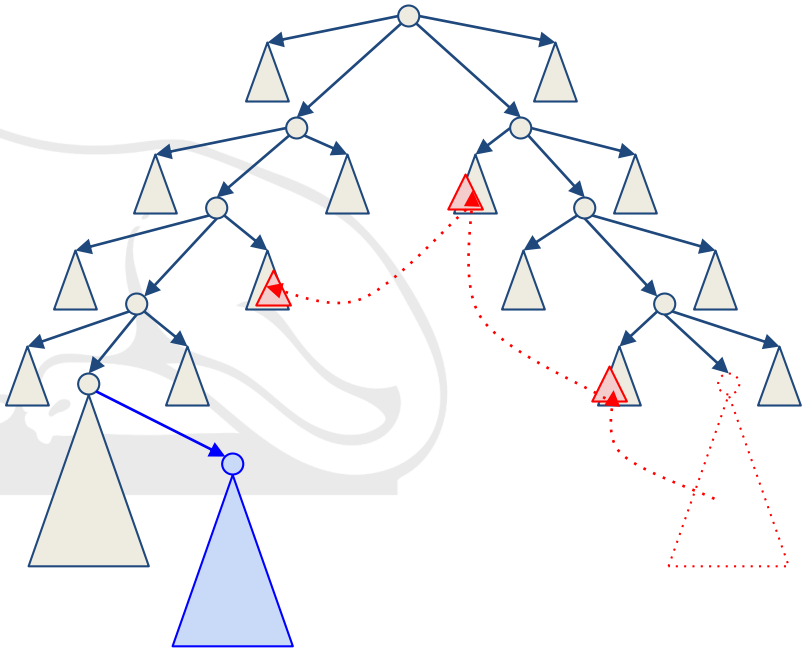
deg(v) charged when reachable from
a new vertex



Incremental Depth First Search Trees (DAG)

Non-Tree edges? Forward Edges, Cross Edges [R-> L]
Reachability? Ancestor to Descendant + R-> L
Insertion of Edge(u,v)? Cross L->R
Detection Top[u] > Top[v]
Affected Vertices? Top[u] > **Top[x]** > Top[v]

Analysis:
Set visit[-] O(n)
Update Topological Order? O(n)
Partial DFS
=> Sum of Degree of new T(v)
deg(v) charged when reachable from
a new vertex



Total Cost= O(n) ∑ deg(v) = O(mn)
Amortized O(n)

Question

Think about **incremental maintenance of DFS** trees.

Directed acyclic graph **O(mn)**

Undirected (Ensure ***monotonic fall)***

Directed graph

(Think of possible difficulty)

Paper Reference

Franciosa et al. 94

Incremental DFS in Directed Graphs
Ponder over extending the algorithm for DAG's to directed graphs.

What needs to be modified?
Where does the analysis vary?